

LLM과 Transformer

실습 가이드

반 4반

번호 U117

이름 박인우

01 실습 배포본 구성

- **02_Attention.ipynb** · 셀프 어텐션 원리 실습
- **Transformer.ipynb** · MiniGPT 통합 구현 실습

02 실습 목표

02_Attention에서 학습한 셀프 어텐션의 수학적 원리가 실제 텍스트 생성 언어 모델(MiniGPT) 구현체에서 어떻게 확장되었는지 5가지 핵심 포인트를 중심으로 분석합니다. 학습한 내용, 실습한 사항, 트러블 슈팅 사항, 느낀점 등을 포함하여 작성할 수 있습니다.

03 분석 내용

1. 다음에 제시한 주제에 대하여 **300~500**자 이내로 본인의 생각을 정리하여 작성하세요

주제 AI 시대에 접어들며 소프트웨어 개발은 많은 변화를 겪게 되었습니다. 소프트웨어 개발에 있어 AI를 사용하는 것에 대한 개인적인 입장을 서술하세요

개발자들은 ai를 사용해서 빠르게 업무를 할 수 있고 자동화까지 할 수 있지만, 이 ai를 사용할때 도메인 지식, 엔지니어링 관점에서 충분히 살펴보고 ai를 잘 통제하고 원하는 답변을 유도를 잘해야 ai를 잘 활용할 수 있다

안 그러면 ai가 그럴듯한 답변을 생성해내게되서 이상한 답변(근거가 없는)을 도출하게 되거나, 쓸데없이 토큰을 많이 잡아먹게 된다(비용증가).

따라서 개발자들은 ai가 만든 답변들을 자신의 지식들을 기반으로 오류가 있는지 없는지 충분히 검토도 해야하며, ai한테 충분한 데이터와 적절한 context를 먹이고, 명확한 요구사항 및 제약사항, 산출물의 적절한 예시 등까지 설계하는게 중요할듯

또 갈수록 단순한 코딩능력보다는 도메인 지식과 엔지니어링 역량을 기반으로 고객들의 페인포인트를 파악해서 문제를 해결해줄 수 있는 설계능력이 훨씬 중요해질 것 같다.

2. SimpleAttention 내 Q, K, V 개별 선언 방식과 역할 분담

비교 대상 기초 실습의 개별 가중치 선언 (W_{query} , W_{key} , W_{value}) vs SimpleAttention 구현체

분석 내용

기초 실습에서 학습한 Q, K, V 생성 과정과 통합 코드의 구현 방식을 비교하여 분석합니다.

Query, Key, Value의 생성부터 Attention Score 계산, Softmax, Value 결합까지의 전체 흐름을 코드와 연결하여 분석합니다.

기초 실습의 Self-Attention이 SimpleAttention에서 어떤 형태로 확장되었는지 자신의 관점에서 설명합니다.

Q, K, V를 만들기 위한 가중치가 각각 별도로 선언된다.

```
W_query = torch.nn.Parameter(torch.rand(d_in, d_out))
```

```
W_key = torch.nn.Parameter(torch.rand(d_in, d_out))
```

```
W_value = torch.nn.Parameter(torch.rand(d_in, d_out))
```

그리고 입력 x 에 각각 다른 가중치 행렬을 곱해 Q, K, V를 생성한다

```
queries = x @ W_query
```

```
keys = x @ W_key
```

```
values = x @ W_value
```

즉 입력 데이터는 동일하지만 세 개의 서로 다른 선형 변환을 거치면서 역할이 다른 표현으로 바뀜

Query(Q): 현재 토큰이 다른 토큰에서 어떤 정보를 찾고 싶은지를 표현

Key(K): 각 토큰이 어떤 정보를 가지고 있는지, 즉 검색을 위한 특징을 표현

Value(V): 실제로 Attention 결과에 전달할 정보를 표현

Q, K 로 유사도를 계산해 v를 어느정도로 가져올지 결정한다.

전체 흐름은

```
queries = x @ W_query
```

```
keys     = x @ W_key
```

```
values   = x @ W_value
```

```
attn_scores = queries @ keys.T
```

```
attn_weights = torch.softmax(  
    attn_scores / keys.shape[-1]**0.5,  
    dim=-1  
)
```

```
context_vec = attn_weights @ values
```

다음으로 가중치들을 클래스로 이동 시킨다

```
class SelfAttention_v1(nn.Module):
```

```
    def __init__(self, d_in, d_out):
```

```
        super().__init__()
```

```
        self.W_query = nn.Parameter(torch.rand(d_in, d_out))
```

```
        self.W_key    = nn.Parameter(torch.rand(d_in, d_out))
```

```
        self.W_value  = nn.Parameter(torch.rand(d_in, d_out))
```

그리고 **foward**안에서,

```
keys = x @ self.W_key
queries = x @ self.W_query
values = x @ self.W_value

attn_scores = queries @ keys.T

attn_weights = torch.softmax(
    attn_scores / keys.shape[-1]**0.5,
    dim=-1
)

context_vec = attn_weights @ values
```

를 수행한다.

즉 셀에서 직접 수행하던 계산을 **mm.module**을 이용해서 안으로 캡슐화

그 다음은 그 다음 구현에서는 직접 만든 **nn.Parameter** 대신:

```
self.W_query = nn.Linear(d_in, d_out, bias=qkv_bias)
self.W_key    = nn.Linear(d_in, d_out, bias=qkv_bias)
self.W_value  = nn.Linear(d_in, d_out, bias=qkv_bias)
```

를 사용한다.

따라서 계산도

```
queries = self.W_query(x)
keys = self.W_key(x)
values = self.W_value(x)
```

그 다음, **Q,K,V**는 그대로 유지한 채, 규모가 커진다. 이제 두가지 기능이 추가된다.

1. Causal Attention(+ Batch 처리 + 미래 Token Mask + Dropout)
2. MultiHeadAttention

특히 **GPT**는 다음 토큰을 예측해야 하므로 미래 토큰을 미리 볼 수 없다. 그래서

```
attn_scores.masked_fill_(mask_bool, -torch.inf)
```

로 **Attention score**를 $-\infty$ 로 만든 뒤 **Softmax**를 적용한다.

배치 처리는 여러 샘플을 한 번에 계산해서 **GPU** 병렬처리를 효율적으로 활용할 수 있고, 샘플 하나씩 학습하는 것보다 **gradient**가 덜 흔들려 학습이 안정적이다.

Dropout은 학습 중 일부 연결이나 값을 랜덤하게 꺼서 특정 패턴에만 과도하게 의존하는 걸 막고 과적합을 줄여 일반화 성능을 높인다.

MultiHeadAttention은 만들어진 벡터를 여러 **head**로 나눈다.

embedding dimension = 768, head = 12

$768 / 12 = 64$ 이므로,

Q [B, T, 768]

K [B, T, 768]

V [B, T, 768]

↓ 분할

[B, 12, T, 64]

으로 나뉜다.

각 **head**가 독립적으로:

$QK^T \rightarrow \text{Mask} \rightarrow \text{Softmax} \rightarrow \text{Attention Weight} \times V$

을 수행하고 마지막에 다시 합친다.

Multi-Head Attention은 하나의 **Attention**만 보는 게 아니라 여러 **head**가 각각 다른 관계나 특징을 동시에 볼 수 있어서 다양한 문맥 관계를 병렬로 학습할 수 있다.

3. 베셀 보정 없는 층 정규화 (LayerNorm)와 학습 안정성

비교 대상 기초 실습에서의 정규화 개념 vs LayerNorm 클래스 구현 (unbiased=False)

분석 내용

통합 코드에서 직접 구현한 *LayerNorm*의 계산 과정과 주요 구성요소를 분석합니다.

mean, *var*, *eps*, *scale*, *shift*가 각각 어떤 역할을 하는지 코드와 연결하여 설명합니다.

기초적인 정규화 개념이 *Transformer* 구조 안에서 어떤 의미를 갖는지 자신의 관점에서 분석합니다.

- 비교 대상: 기초 실습에서의 정규화 개념 vs LayerNorm 클래스 구현 (unbiased=False)
- 학생 분석 관점:
 - 통합 코드에서 직접 구현한 LayerNorm의 계산 과정과 주요 구성요소를 분석합니다.
 - mean, var, eps, scale, shift가 각각 어떤 역할을 하는지 코드와 연결하여 설명합니다.
 - 기초적인 정규화 개념이 Transformer 구조 안에서 어떤 의미를 갖는지 자신의 관점에서 분석합니다.

```
class LayerNorm(nn.Module):  
  
    def __init__(self, emb_dim):  
        super().__init__()  
  
        self.eps = 1e-5  
  
        self.scale = nn.Parameter(torch.ones(emb_dim))  
  
        self.shift = nn.Parameter(torch.zeros(emb_dim))  
  
  
    def forward(self, x):  
  
        mean = x.mean(dim=-1, keepdim=True)  
  
        var = x.var(dim=-1, keepdim=True, unbiased=False)  
  
        norm_x = (x - mean) / torch.sqrt(var + self.eps)  
  
        return self.scale * norm_x + self.shift
```

정규화는 값들의 평균을 0 근처, 분산을 1 근처로 맞춰 값의 크기를 일정하게 만드는 것이 핵심

```
x.shape = [batch, tokens, emb_dim] = [2, 6, 768]
```

이러면, `mean = x.mean(dim=-1, keepdim=True)` 의 `dim=-1`은 마지막 차원인 768개의 **embedding feature**를 기준으로 평균을 계산

```
norm_x = (x - mean) / torch.sqrt(var + self.eps)
```

으로 정규화 한다.

요소	코드	역할
mean	<code>x.mean(dim=-1, keepdim=True)</code>	각 토큰의 feature 평균을 계산하고 중심을 0으로 이동
var	<code>x.var(..., unbiased=False)</code>	각 토큰 feature 값들이 얼마나 퍼져 있는지 계산
eps	<code>1e-5</code>	분산이 0이거나 매우 작을 때 0으로 나누는 문제 방지
scale	<code>nn.Parameter(torch.ones(emb_dim))</code>	정규화된 값을 모델이 필요한 크기로 다시 조절하는 학습 파라미터 γ
shift	<code>nn.Parameter(torch.zeros(emb_dim))</code>	정규화된 값을 필요한 위치로 이동시키는 학습 파라미터 β

`unbiased=False`는 베셀 보정을 하지 않겠다는 뜻.

베셀 보정(Bessel's correction)은 유한 표본으로부터 모집단 분산을 추정할 때 발생하는 편향(bias)을 제거해 준다. 딥러닝과 대규모 언어 모델(LLM)에서 베셀 보정(분모를 $n-1$ 대신 n 으로 사용하는 것)을 엄격하게 적용하지 않는 이유는 모델이 다루는 데이터의 크기가 워낙 크기 때문에 보정 여부가 주는 수학적 차이가 거의 없고, 기존에 학습된 모델 구조와 가중치를 그대로 가져다 쓰기 위한 실용적인 이유 때문.

왜 **LayerNorm**을 하나?

llm모델 같은경우 층이 굉장히 많다(GPT-2 Small은 Transformer Block이 12). 중간 활성값의 스케일이 계속 불안정하게 변하면 깊은 층으로 갈수록 학습이 어려워질 수 있음. **LayerNorm**이 각 토큰의 표현을 일정한 범위로 정리해주면서 학습을 안정화시키는 역할을 한다.

특히, **TransformerBlock** 을 보면,

```
shortcut = x
```

```
x = self.norm1(x)
```

```
x = self.att(x)
```

```
x = self.drop_shortcut(x)
```

```
x = x + shortcut
```

다시 **shortcut = x**

```
x = self.norm2(x)
```

```
x = self.ff(x)
```

```
x = self.drop_shortcut(x)
```

```
x = x + shortcut
```

하고 마지막에

```
self.final_norm = LayerNorm(cfg["emb_dim"])
```

한번 더 정규화하는 모습을 볼 수 있다.

참고로 레지듀얼 구조(입력값을 더해서 잔차만 학습시키는 구조)는 그레디언트 소실문제를 크게 완화시켜준다.

부록. 배치 노름 하고의 차이

구분		BatchNorm	LayerNorm
정규화 기준	배치에 있는 여러 샘플을 기준	각 샘플/토큰 내부의 feature 를 기준	
통계 계산	같은 feature 의 배치 평균·분산	한 토큰의 embedding 차원 평균·분산	
배치 크기 영향	있음	거의 없음	

Train / Eval	동작이 다름. 학습 시 batch 통계, 추론 시 running 통계 사용	기본적으로 동일한 방식
Transformer/LM	일반적으로 잘 사용하지 않음	매우 많이 사용
장점	CNN 등에서 안정적이고 효율적	문장 길이·배치 크기가 달라도 안정적

Transformer에서는 문장마다 길이가 다를 수 있고, 추론할 때는 **batch_size=1**일 수도 있기 때문에 배치 전체 통계에 의존하지 않는 **LayerNorm**이 더 적합

BatchNorm은 배치 내 여러 샘플의 통계를 이용하기 때문에 배치 크기에 영향을 받는 반면, **LayerNorm**은 각 토큰의 **feature** 차원에서 독립적으로 평균과 분산을 계산하므로 배치 크기에 의존하지 않으며, 이러한 특성 때문에 Transformer와 언어 모델의 학습에 적합하다.

4. 활성화 함수와 피드포워드 신경망 (GELU 및 FeedForward)

비교 대상 단순 선형 변환 레이어 vs GELU와 FeedForward로 구성된 블록

분석 내용

*FeedForward*의 구조와 차원 변화 과정을 코드와 연결하여 분석합니다.

*GELU*가 전체 구조에서 어떤 역할을 하는지 설명합니다.

*Attention*과 *FeedForward*가 각각 어떤 역할을 수행하는지 비교하여 분석합니다.

```
class FeedForward(nn.Module):
    def __init__(self, cfg):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(cfg["emb_dim"], 4 * cfg["emb_dim"]),
```

```
        GELU(),
        nn.Linear(4 * cfg["emb_dim"], cfg["emb_dim"]),
    )
```

```
def forward(self, x):
    return self.layers(x)
```

emb_dim = 768 이므로 차원 변화를 살펴 보면,

768

↓ Linear

3072

↓ GELU

3072

↓ Linear

768

한 번 4배 넓은 공간으로 확장해서 다양한 특징 조합을 만든 뒤 다시 원래 **embedding** 크기로 줄이는 구조

왜 그냥 **Linear** 두 개를 쓰지 않고 **GELU**를 넣는가?

Linear(768, 3072) Linear(3072, 768) 만 연속으로 사용하면 둘 다 선형변환이므로 결국 하나의 선형변환과 같은 형태로 합쳐질 수 있음.

$$W_2(W_1x) = W'x$$

그래서 단순히 **feature**를 다시 섞는 것 이상의 복잡한 관계를 표현하기 어렵다.

중간에 **GELU()** 같은 비선형 함수를 넣으면,

Linear

↓

비선형 변환

↓

Linear

이 되면서 단순 선형변환으로 합칠 수 없게 되고, 모델이 훨씬 복잡한 패턴을 학습할 수 있게 됨.

```
class GELU(nn.Module):
    def forward(self, x):
        return 0.5 * x * (1 + torch.tanh(
            torch.sqrt(torch.tensor(2.0 / torch.pi)) *
            (x + 0.044715 * torch.pow(x, 3))
        ))
```

GELU, ReLU 비교

ReLU

음수 → 전부 0

양수 → 그대로

GELU

음수 → 무조건 0으로 자르지 않고 부드럽게 조절

양수 → 크기에 따라 부드럽게 통과

Attention과 FeedForward의 역할 차이

Attention은 토큰들 사이의 관계를 계산해서 다른 토큰의 정보를 현재 토큰에 섞어준다.

Attention = 토큰 간 정보 교환(Token Mixing)

FeedForward

FeedForward는 다른 토큰과 직접 비교하지 않는다.

Attention을 거쳐 이미 문맥 정보가 섞인 각 토큰 벡터를 각각 독립적으로

768 → 3072 → GELU → 768

변환함. 즉, FeedForward = 각 토큰 내부의 feature를 가공(Feature Mixing)

Transformer는 Attention을 통해 문맥 정보를 모으고, FeedForward를 통해 그 정보를 비선형적으로 가공(GELU)하는 두 과정을 반복

5. 숏컷 연결(Residual Connection)과 TransformerBlock의 결합

비교 대상 단일 어텐션 모듈 vs TransformerBlock 구조 ($x = x + \text{self.att}(\dots)$)

분석 내용

*TransformerBlock*을 구성하는 주요 요소와 이들의 연결 관계를 분석합니다.

Attention, *LayerNorm*, *FeedForward*, *Residual Connection*이 어떤 순서와 방식으로 결합되어 있는지 코드와 연결하여 설명합니다.

기초 실습의 단일 *Attention*이 하나의 *TransformerBlock*으로 확장되면서 어떤 구조적 변화가 발생했는지 분석합니다.

```
class TransformerBlock(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        self.att = MultiHeadAttention(...)
        self.ff = FeedForward(cfg)
        self.norm1 = LayerNorm(cfg["emb_dim"])
        self.norm2 = LayerNorm(cfg["emb_dim"])
        self.drop_shortcut = nn.Dropout(cfg["drop_rate"])

    def forward(self, x):
        # Attention
        shortcut = x
        x = self.norm1(x)
```

```

    x = self.att(x)

    x = self.drop_shortcut(x)

    x = x + shortcut

    # FeedForward

    shortcut = x

    x = self.norm2(x)

    x = self.ff(x)

    x = self.drop_shortcut(x)

    x = x + shortcut

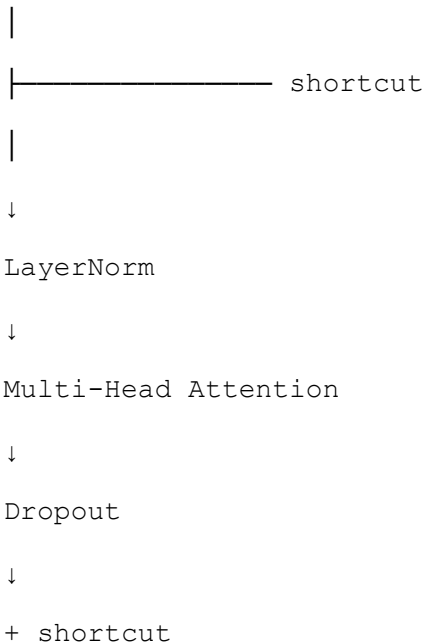
return x

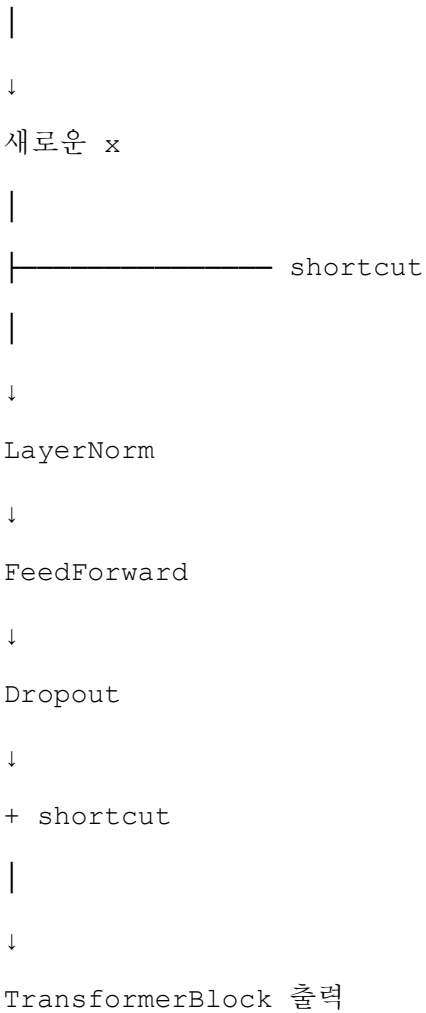
```

레지듀얼 : 이전 입력을 이후 출력에 직접 더해 **gradient**가 깊은 네트워크를 통과할 수 있는 경로를 만드는 구조

전체 흐름

입력 x





크게 **Attention** 부분과 **FeedForward** 부분에 각각 **Residual Connection**이 하나씩 들어감.

Attention

```
shortcut = x
x = self.norm1(x)
x = self.att(x)
x = self.drop_shortcut(x)
x = x + shortcut
```

각각의 역할은

LayerNorm → 입력 크기 안정화

Attention → 토큰끼리 문맥 정보 교환

Dropout → 과적합 방지

+ shortcut → 원래 입력 정보 보존

이 x 를 다시 **FeedForward**에 집어넣음

```
x = self.norm2(x)
```

```
x = self.ff(x)
```

```
x = self.drop_shortcut(x)
```

```
x = x + shortcut
```

Residual Connection은 왜 필요한가?

기존 정보 x + 이번 **Layer**가 새롭게 계산한 정보 구조로 됨.

이 뜻은, 모델이 기존 표현을 유지하면서 필요한 부분만 수정하는 방식

무엇보다 역전파를 할때, **shortcut**을 통해 직접적인 **gradient** 경로가 생겨 깊은 모델에서 **gradient**가 지나가기 쉬워진다 즉, 그레디언트 소실 문제를 완화해준다

기초 **Attention**에서 **TransformerBlock**으로 무엇이 확장됐나

입력

↓

Q, K, V

↓

Attention

↓

Context Vector

에서, 실제 **TransformerBlock**은

기초 Self-Attention

↓

Multi-Head Attention으로 확장

```

      +
LayerNorm
      +
Dropout
      +
Residual Connection
      +
FeedForward + GELU
      ↓
TransformerBlock

```

으로 확장

다시 정리해보면,

구성 요소	역할
LayerNorm	값의 스케일 안정화
Multi-Head Attention	토큰 간 문맥 정보 결합
Dropout	과적합 방지
Residual Connection	기존 정보 보존 + gradient 전달
FeedForward	각 토큰 내부 feature를 비선형적으로 가공

그리고 GPT에서는 이 TransformerBlock을 여러층 쌓아서 깊은 모델을 만듦

6. 전체 LLM 아키텍처 파이프라인의 완성 (MiniGPT와 CONFIG)

비교 대상

단품 텐서 연산 검증 단계 vs MiniGPT 통합 클래스

(tok_emb, pos_emb, trf_blocks, final_norm, out_head)

분석 내용

입력 문장이 *Token ID*로 변환된 이후 모델을 통과하는 전체 데이터 흐름을 분석합니다.

*Token Embedding, Position Embedding, TransformerBlock, Final LayerNorm, Output Head*의 연결 관계를 코드와 함께 분석합니다.

현재 설정값(*vocab_size, context_length, emb_dim* 등)이 모델의 입력과 출력 구조에 어떤 영향을 미치는지 분석합니다.

최종 *Logit*이 생성되고 다음 단어 후보를 선택하는 과정까지 포함하여 전체 *MiniGPT Pipeline*의 동작을 자신의 관점에서 설명합니다.

설정값 :

```
GPT_CONFIG_124M = {
    "vocab_size": 50257,
    "context_length": 1024,
    "emb_dim": 768,
    "n_heads": 12,
    "n_layers": 12,
    "drop_rate": 0.1,
    "qkv_bias": False
}
```

```
class GPTModel(nn.Module):
    def __init__(self, cfg):
        super().__init__()

        self.tok_emb = nn.Embedding(
            cfg["vocab_size"], cfg["emb_dim"]
        )
```

```
self.pos_emb = nn.Embedding(
    cfg["context_length"], cfg["emb_dim"]
)

self.drop_emb = nn.Dropout(cfg["drop_rate"])

self.trf_blocks = nn.Sequential(
    *[TransformerBlock(cfg)
      for _ in range(cfg["n_layers"])]
)

self.final_norm = LayerNorm(cfg["emb_dim"])

self.out_head = nn.Linear(
    cfg["emb_dim"],
    cfg["vocab_size"],
    bias=False
)
```

Token Embedding

```
tok_embeddings = self.tok_emb(in_idx)
```

`nn.Embedding(50257, 768)`이라는 학습 가능한 임베딩 테이블이 만들어짐. 각 **Token ID**가 **768**차원 벡터 하나로 변환

Position Embedding

Token Embedding만 있으면 모델은 토큰이 무엇인지는 알 수 있지만 몇 번째 위치인지는 알 수 없음. 위치 임베딩도 각 토큰마다 **768**차원 만듦.

```
x = tok_embeddings + pos_embeddings
```

로 무슨 토큰인지, 어디에 있는지 함께 저장

TransformerBlock 반복

```
x = self.trf_blocks(x)
```

설정 값에 따라 TransformerBlock을 12번 통과

각 TransformerBlock 안에서는 앞에서 본

LayerNorm

↓

Multi-Head Causal Attention

↓

Residual Connection

↓

LayerNorm

↓

FeedForward + GELU

↓

Residual Connection

과정이 수행되어 Attention은 토큰 간 문맥 관계를 결합하고, FeedForward는 각 토큰의 feature를 가공

Final LayerNorm 12개 TransformerBlock을 모두 통과한 뒤

```
x = self.final_norm(x)
```

을 적용, 마지막 스케일을 안정적으로 정규화함.

Output Head

```
logits = self.out_head(x)
```

이건

```
nn.Linear(768, 50257, bias=False)
```

이므로, 각 토큰의 768차원 hidden vector를 Vocabulary 전체 50,257개에 대한 점수로 바꿈.

```
[B, T, 768]
```

↓ Output Head

`[B,T,50257]`

즉, GPT 모델이 입력 Token ID를 받아 Vocabulary 크기만큼의 Logit을 출력하고, 다시 Token ID와 텍스트로 변환하는게 전체 흐름임.

다음 Token 선택

```
logits = model(idx_cond)
```

으로

`[B,T,50257]`

를 얻고

```
logits = logits[:, -1, :]
```

로 마지막 위치의 예측만 선택

`[B,T,50257]`

↓

`[B,50257]`

그다음

```
probas = torch.softmax(logits, dim=-1)
```

으로 Logit을 확률로 변환하고

```
idx_next = torch.argmax(
```

```
    probas,
```

```
    dim=-1,
```

```
    keepdim=True
```

```
)
```

가장 확률이 높은 Token ID를 선택

50257개 Logit

↓

Softmax

↓

50257개 확률

↓

argmax

↓

다음 Token ID 하나

그 **Token**을 기존 입력 뒤에 붙인다.

```
idx = torch.cat((idx, idx_next), dim=1)
```

그리고 다시 모델에 넣어

Every effort moves you

↓

다음 token 생성

↓

Every effort moves you forward

↓

다시 입력

↓

다음 token 생성

↓

Every effort moves you forward ...

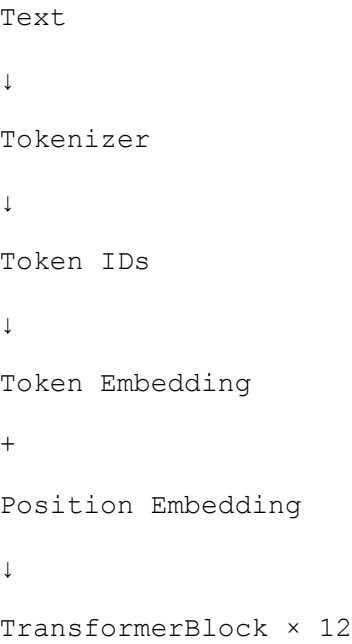
총정리를 하면,

각 토큰을 의미+위치 벡터로 만든 뒤 **Transformer**를 여러 층 통과시켜 문맥이 반영된 표현으로 만들고, 마지막 표현을 **Vocabulary** 전체의 점수로 변환하여 다음 토큰 하나를 선택하는 과정을 반복한다.

CONFIG 값들이 구조에 미치는 영향

설정	의미	모델에 미치는 영향
<code>vocab_size=50257</code>	사용할 Token 종류 수	Embedding 행 개수와 Output Logit 개수 결정
<code>context_length=1024</code>	한 번에 볼 최대 Token 수	Position Embedding 크기와 최대 문맥 길이 결정
<code>emb_dim=768</code>	Token 표현 차원	모델 내부 hidden vector 크기 결정
<code>n_heads=12</code>	Attention Head 수	768차원을 12개 Head로 분할, Head당 64차원
<code>n_layers=12</code>	TransformerBlock 수	모델의 깊이 결정
<code>drop_rate=0.1</code>	Dropout 비율	학습 중 과적합 방지
<code>qkv_bias=False</code>	Q/K/V Linear의 bias 여부	Q/K/V Projection 구조 결정

총 흐름 정리,이해



```
└ Attention
└ LayerNorm
└ FeedForward
└ Residual
↓
Final LayerNorm
↓
Output Head
↓
Vocabulary Logits
↓
다음 Token 선택
↓
다시 입력
↓
반복
```

04 최종 성찰 및 트러블 슈팅

가장 중요하게 이해한 개념 및 해석

Attention은 Q, K, V를 이용하여 토큰 간 관계를 계산하고 필요한 문맥 정보를 결합하는 역할을 하고, **FeedForward**는 **Attention**을 통해 문맥이 반영된 각 토큰의 **feature**를 비선형적으로 변환하고 가공하는 역할을 한다. 여기에 **LayerNorm**은 각 단계의 값의 범위를 안정화하고, **Residual Connection**은 기존 정보를 보존하면서 새로운 정보를 추가하고 **gradient**가 깊은 네트워크를 통과하기 쉽게 만든다.

LLM은 단순히 “다음 단어를 맞히는 모델”이 아니라, 각 토큰을 임베딩한 뒤 **Attention**으로 문맥을 구성하고 **FeedForward**로 그 표현을 가공하는 과정을 여러 층 반복하여 다음 토큰의 확률을 계산하는 구조

문장

↓

Tokenizer

↓

Token ID

↓

Token Embedding + Position Embedding

↓

TransformerBlock 반복

└─ LayerNorm

└─ Multi-Head Causal Attention

└─ Residual Connection

└─ LayerNorm

└─ FeedForward + GELU

└─ Residual Connection

↓

Final LayerNorm

↓

Output Head

↓

Vocabulary Logit

↓

다음 Token 선택

↓

다시 입력하여 반복

부록 . 궁금했던점

Q. 쿼리키밸류로 나누는 이유가 멀까 그냥 단순히 **w** 하나만 두고 내적곱해서 유사도 하는거랑 실제로 어떤 차이가 있는지 관련 실험이 있어?

A.

*****Q/K/V 세 개가 정말 필요한가? 그냥 `W` 하나로 임베딩을 변환한 뒤 서로 내적하면 안 되나?*****는 실제로 연구된 주제고, 결론부터 말하면:

> ****가능하다. 실제로 꽤 잘 된다.****

> 다만 Q/K/V를 분리하면 특히 ****방향성** 있는 관계와 ‘검색 **vs** 전달’ 역할을 더 자유롭게 학습******할 수 있어서 표준 Transformer는 세 개를 쓴다.

오늘 자료는 표준 구조인 $Q=XW_Q$, $K=XW_K$, $V=XW_V$ 를 설명하고 있고, 아래 실험 얘기는 자료 밖의 연구 결과야.

1. 네가 말한 `W` 하나` 방식부터 보면

각 토큰 `x`를 하나의 행렬로 변환해서:

$$z_i = x_i W$$

$$z_j = x_j W$$

둘의 유사도를

$$\text{score}(i,j) = z_i \cdot z_j$$

로 구하면 돼.

충분히 Attention 비슷한 걸 만들 수 있어.

그런데 중요한 특성이 생겨.

$$z_i \cdot z_j = z_j \cdot z_i$$

즉,

$$\text{score}(i \rightarrow j) = \text{score}(j \rightarrow i)$$

가 돼.

****관계가 대칭적****이 되는 거야.

2. Q와 K를 따로 두면 이게 달라짐

표준 Attention은:

$$q_i = x_i WQ$$

$$k_j = x_j WK$$

$$\text{score}(i \rightarrow j) = q_i \cdot k_j$$

반대 방향은:

$$\text{score}(j \rightarrow i) = q_j \cdot k_i$$

이고 `WQ ≠ WK`니까

$$\text{score}(i \rightarrow j) \neq \text{score}(j \rightarrow i)$$

여도 아무 문제가 없어.

이게 생각보다 중요해.

예를 들어:

The cat drank milk because it was thirsty.

`it` 입장에서:

it → cat

을 강하게 찾아야 할 수 있어.

그렇다고 반대로

cat → it

도 정확히 같은 관계 강도를 가져야 할 이유는 없잖아.

자연어에는 이런 방향성이 엄청 많아.

대명사 → 선행사

동사 → 주어

동사 → 목적어

수식어 → 피수식어

현재 토큰 → 과거의 관련 토큰

그래서 Q/K 분리는 단순히 ****유사도를 계산****하는 것보다,

> ****A가 찾는 특성과 B가 검색될 특성을 서로 다르게 만들 수 있게 한다****

는 게 훨씬 정확한 설명이야.

수학적으로도 차이가 뚜렷해.

하나의 W면:

$\text{score}(i,j)$

$= x_i W W^T x_j^T$

인데 `WW<sup>T</sup>`는 대칭 행렬이야.

Q/K가 다르면:

$\text{score}(i,j)$

$= x_i WQ WK^T x_j^T$

이고

$WQWK^T$

는 일반적으로 대칭일 필요가 없어.

즉 ****모델이 학습할 수 있는 관계의 범위 자체가 넓어진다.****

3. 그러면 V 는 왜 또 따로 있냐

여기서 또 한 번 역할을 분리하는 거야.

Q, K

↓

"누구를 가져올까?"

V

↓

"그 토큰에서 무엇을 가져올까?"

예를 들어 " cat "을 **Attention** 대상으로 선택했다고 해보자.

$K(cat)$ 은

"나는 이런 상황에서 선택될 만한 토큰이다"

를 잘 표현하면 되고,

$V(cat)$ 은

"선택됐을 때 다른 토큰에게 전달할 정보"

를 잘 표현하면 돼.

하나의 W 를 쓰면 같은 벡터가:

① 검색되기 좋아야 하고

② 다른 토큰과 비교하기 좋아야 하고

③ 실제 전달할 정보도 잘 표현해야 함

세 가지 일을 다 해야 해.

Q/K/V를 나누면:

WQ → 찾는 데 최적화

WK → 검색되는 데 최적화

WV → 전달하는 데 최적화

로 각각 학습할 수 있어.

그런데 실제 실험을 해보니 재미있는 결과가 나왔음

네가 의심한 것처럼 ****Q/K/V 3개가 절대적으로 필수는 아니야.****

2025년 NAACL 논문에서는 아예

$Q = K = V$

즉 ****하나의 weight matrix를 세 역할에 공유****하는 BERT를 실험했어.

결과는 Attention 블록의 파라미터를 ****66.53% 줄이면서도****, 작은 GLUE 태스크들에서는 표준 BERT와 경쟁력 있는 성능을 냈고 일부 조건에서는 더 좋은 결과도 보고했어. ([ACL Anthology][1])

즉:

> "W 하나 쓰면 Transformer가 망한다."

는 건 틀린 말이야.

더 흥미로운 실험도 있어. 2024년 연구에서는 ****Q와 K를 얼마나 공유할 수 있는지**** 실험했는데, 완전히 $Q=K$ 로 만들면 Attention이 지나치게 대칭적이 되면서 ****방향 관계 표현이 단순해지고 pretraining loss가 악화****되는 현상을 보고했어. 반면 일부만 공유하면 상당히 많은 파라미터를 공유하면서 성능을 유지할 수 있었어. ([Proceedings of Machine Learning Research][2])

그리고 2026년 QKV projection을 체계적으로 비교한 연구에서는:

① Q, K, V 모두 별도 ← 표준

② Q 별도, K=V 공유

③ Q=K 공유, V 별도

④ Q=K=V 모두 공유

를 비교했어.

특히 언어 모델에서는

Q는 별도

K = V 공유

가 상당히 잘 버텼어. 300M/1.2B 규모 언어모델 실험에서 KV cache를 **50%** 줄이면서 perplexity 악화가 약 **3.1%** 수준이었다고 보고했어. 반대로 `Q=K`처럼 Query와 Key를 묶는 쪽은 **방향성을 잃는 문제가 더 중요**하게 나타났어. ([arXiv][3])

이 결과가 상당히 의미 있어.

Q ↔ K 분리

→ 꽤 중요함

→ "찾는 쪽"과 "찾히는 쪽"의 방향성

K ↔ V 분리

→ 생각보다 덜 중요할 수도 있음

→ 실제로 공유해서 메모리를 줄이는 연구 존재

라는 거니까.

심지어 2021년 **Synthesizer** 연구에서는 아예 `Q·K` 기반 token-to-token 유사도 계산을 없애버린 Attention도 실험했는데 상당수 과제에서 Transformer와 경쟁력 있는 성능을 보였어. 즉 `QK dot-product` 자체도 유일한 정답은 아니라는 결과야. ([Proceedings of Machine Learning Research][4])

즉 **QKV는 Attention이 작동하기 위한 절대적인 수학적 필수조건**이라기보다, 모델에게 역할 분리와 더 큰 표현 자유도를 주는 설계라고 보는 게 제일 좋아.

그리고 최근 연구들을 보면 네 질문처럼 **“사실 세 개까지 필요 없지 않나?”**라는 의문이 실제 Transformer 경량화 연구의 한 방향이 되고 있어. 특히 Q/K 분리의 방향성은 꽤 가치가 있지만, K/V는 경우에 따라 공유할 여지가 있다는 게 재미있는 부분이야.